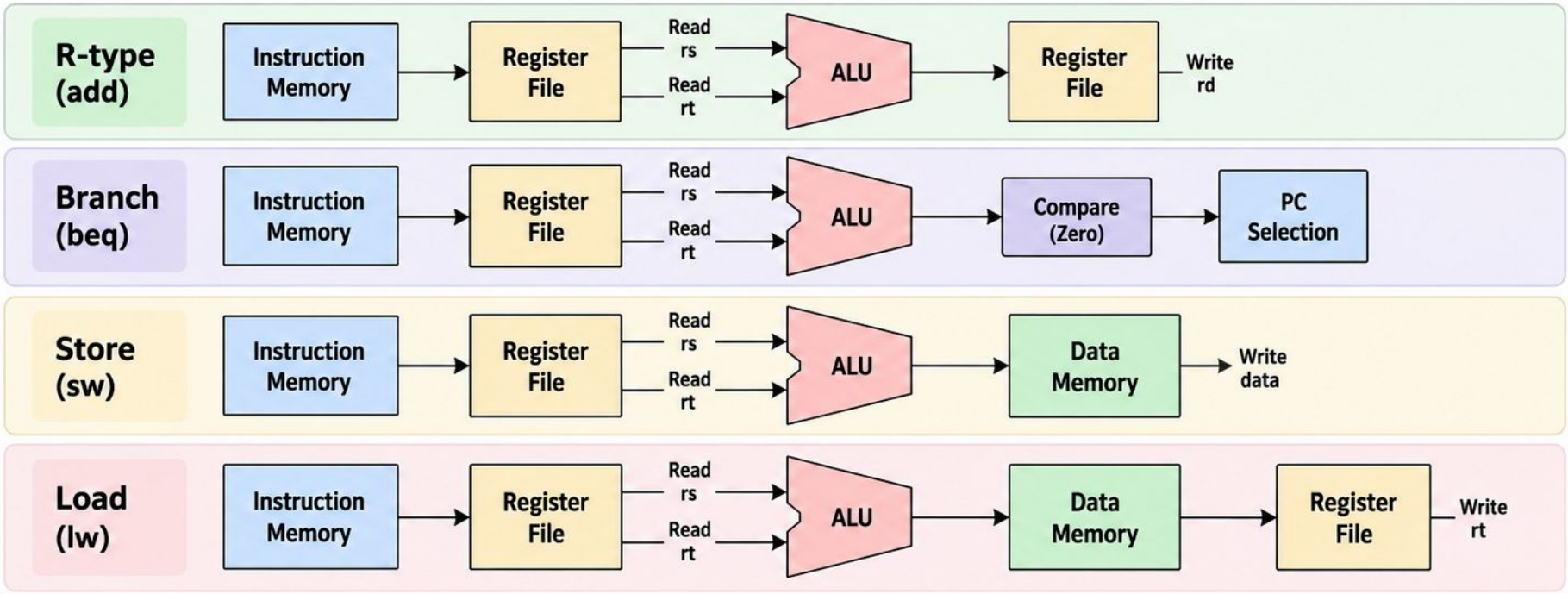
Chapter 3

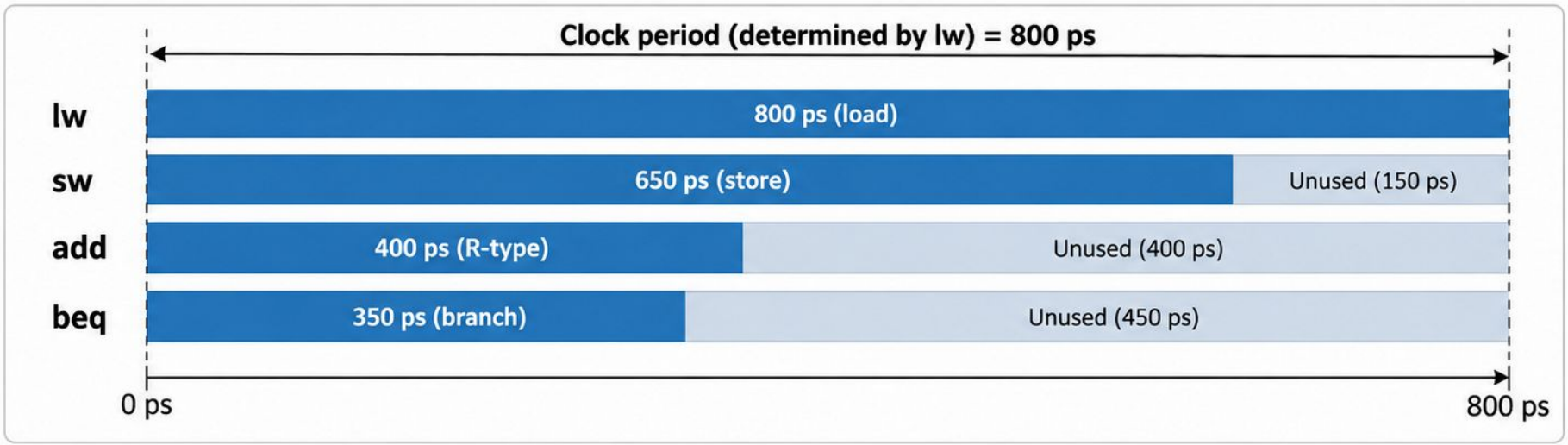
The Processor

Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by pipelining

Performance Issues





Performance Issues

Q1. A single-cycle processor with instruction delays:

Instruction	Delay
R-type (add)	400 ps
Branch (beq)	350 ps
Store (sw)	650 ps
Load (lw)	800 ps

(a) What is the minimum clock period required to execute all four instruction types?

(b) If the processor executes an add instruction, how much time is wasted within that clock cycle?

A1.

(a) Clock Period = 800 ps, determined by the longest instruction delay (lw).

(b) Wasted Time = 800 – 400 = 400 ps for add.

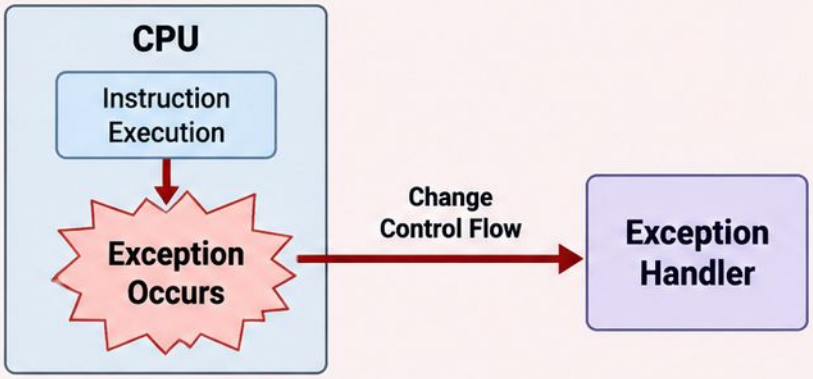
Exceptions and Interrupts

- “Unexpected” events requiring change in flow of control
  - Different ISAs use the terms differently
- Exception
  - Arises within the CPU
    - e.g., undefined opcode, overflow, syscall, …
- Interrupt
  - From an external I/O controller
- Dealing with them without sacrificing performance is hard

What's the Difference

Exception

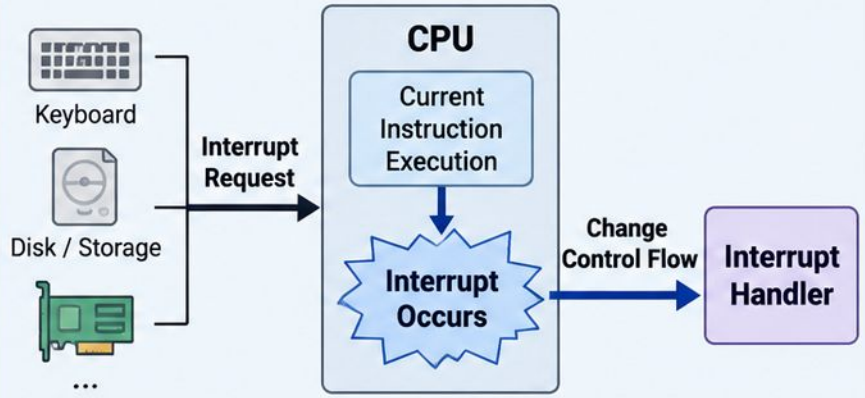
Internal Event (within the CPU)



- Originates within the CPU during instruction execution.
- Examples:
  - Arithmetic overflow (e.g., add)
  - Undefined opcode (invalid instruction)
  - System call (e.g., syscall)

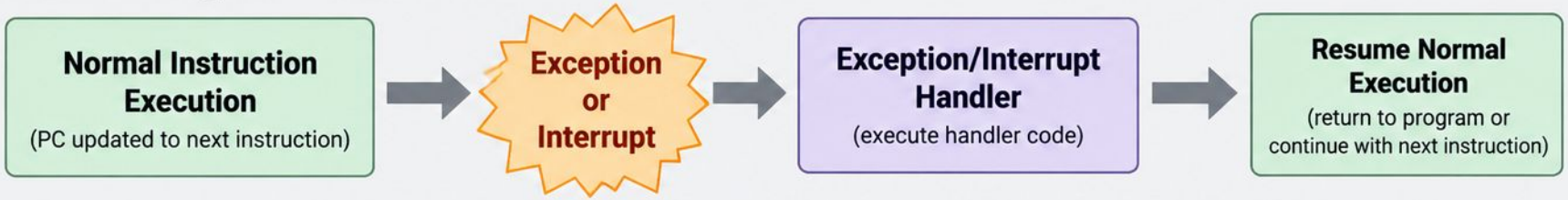
Interrupt

External Event (from an I/O device)



- Originates from an external device or I/O controller.
- Examples:
  - Keyboard input
  - I/O completion (e.g., disk read/write finished)
  - Timer interrupt
  - ...

Effect on Program Execution



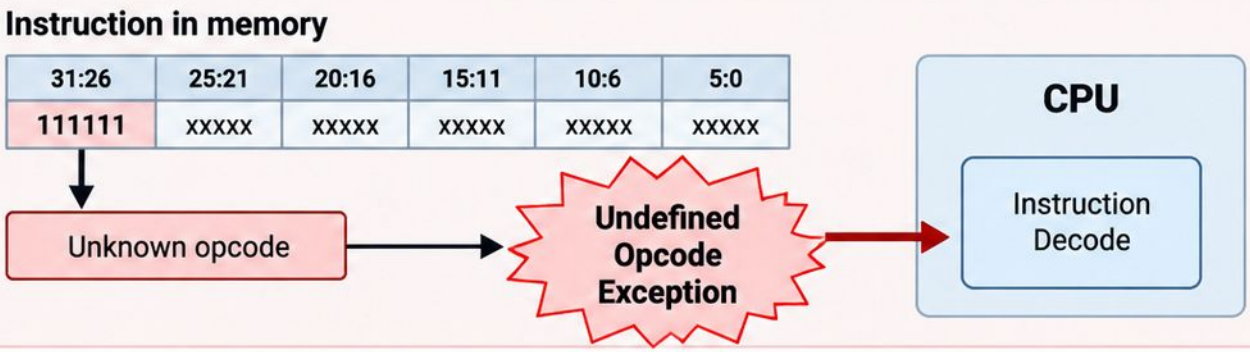
Common Exception Types

- Undefined Opcode: when the CPU encounters a n invalid or unsupported instruction
- Arithmetic Overflow: when a signed arithmetic re sult exceeds the representable range.
- System Call: when a program executes syscall t o request an operating system service.

Common Exception Types

Undefined Opcode

Occurs when the CPU encounters an invalid or unsupported instruction.



Arithmetic Overflow

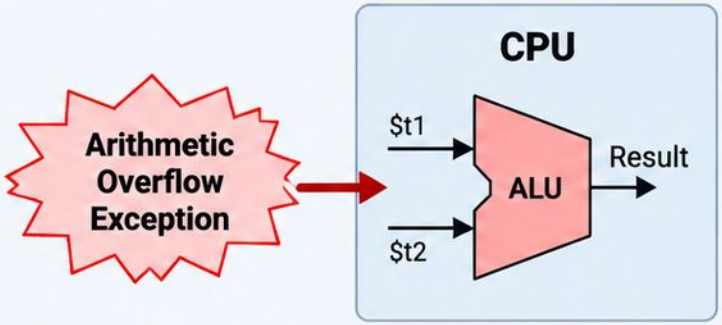
Occurs when a signed arithmetic result exceeds the representable range (e.g., using add).

Example: `add $t0,$t1,$t2`

$\$t1 = 0x7FFFFFFF \quad (2^{31} - 1)$
$\$t2 = 0x00000001 \quad (1)$

$Result = 0x80000000 \quad (2^{31})$

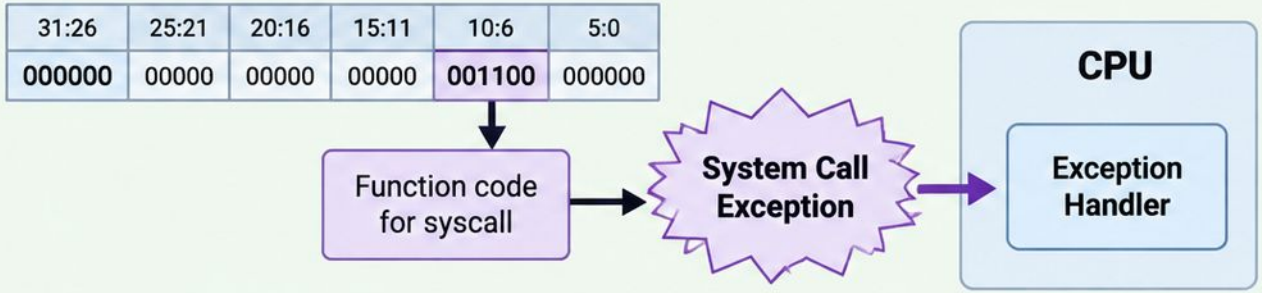
Exceeds 32-bit signed range (-2^{31} to $2^{31} - 1$)



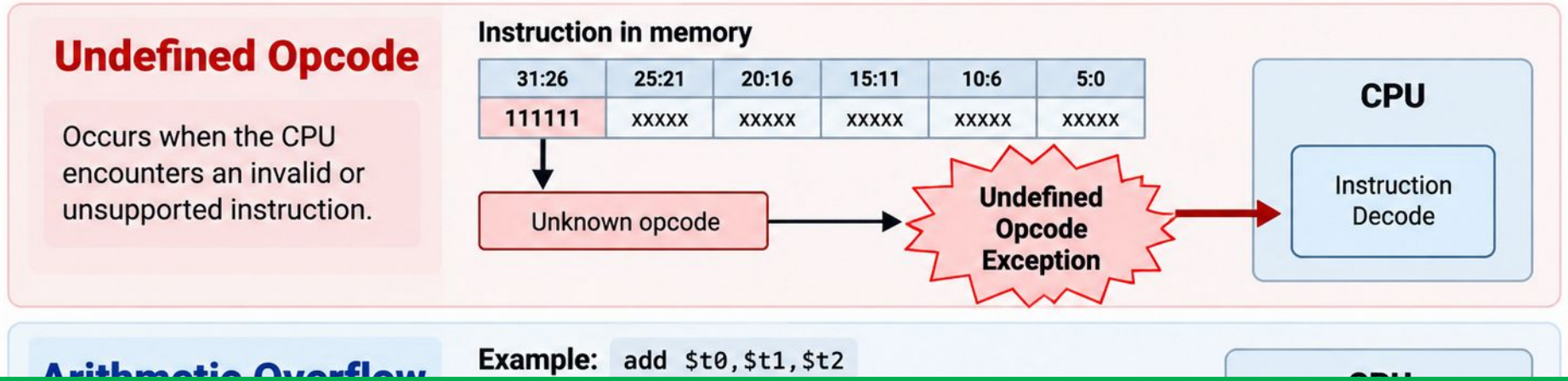
System Call

Occurs when a program executes `syscall` to request an operating system service.

Example: `syscall`



Common Exception Types



Q2. A processor encounters the following events:
Event A: An add instruction produces a signed arithmetic overflow.
Event B: A keyboard controller signals that a key is pressed.
Event C: A program executes a syscall instruction.
Which events are Exceptions, and which are Interrupts?

A2.
Event A: Exception — Arithmetic Overflow.
Event B: Interrupt — External I/O event.
Event C: Exception — System Call.

Handling Exceptions

- In MIPS, exceptions managed by a System Control Coprocessor (CP0)
- Save PC of offending (or interrupted) instruction
  - In MIPS: Exception Program Counter (EPC)
- Save indication of the problem
  - In MIPS: Cause register
  - We'll assume 1-bit
    - 0 for undefined opcode, 1 for overflow
- Jump to handler at 8000 00180

Handling Exceptions

- In MIPS, exceptions managed by a System Control Coprocessor (CP0)

Mechanism	Undefined Opcode	Arithmetic Overflow	System Call
CP0	Manages the excepti on	Manages the exceptio n	Manages the exce ption
EPC	Saves the invalid ins truction address	Saves the overflowing instruction address	Saves the syscall i nstruction address
Cause Register	Records Undefined Opcode	Records Arithmetic O verflow	Records System Call
Handler Address	0x80000180	0x80000180	0x80000180

Handling Exceptions

Q3. processor has the following state:

Address	Instruction
0x00400020	add $t0, $t1, $t2
0x00400024	sub $t3, $t4, $t5
0x00400028	[Unknown Instructio n]
0x0040002C	lw $t6, 0($t7)

EPC =

Cause = 0 (Undefined Opcode)

PC = 0x80000180

Which instruction caused the exception, and what type of problem occurred?

A3.

The instruction at 0x00400028 caused the exception.

Cause = 0: Undefined Opcode.

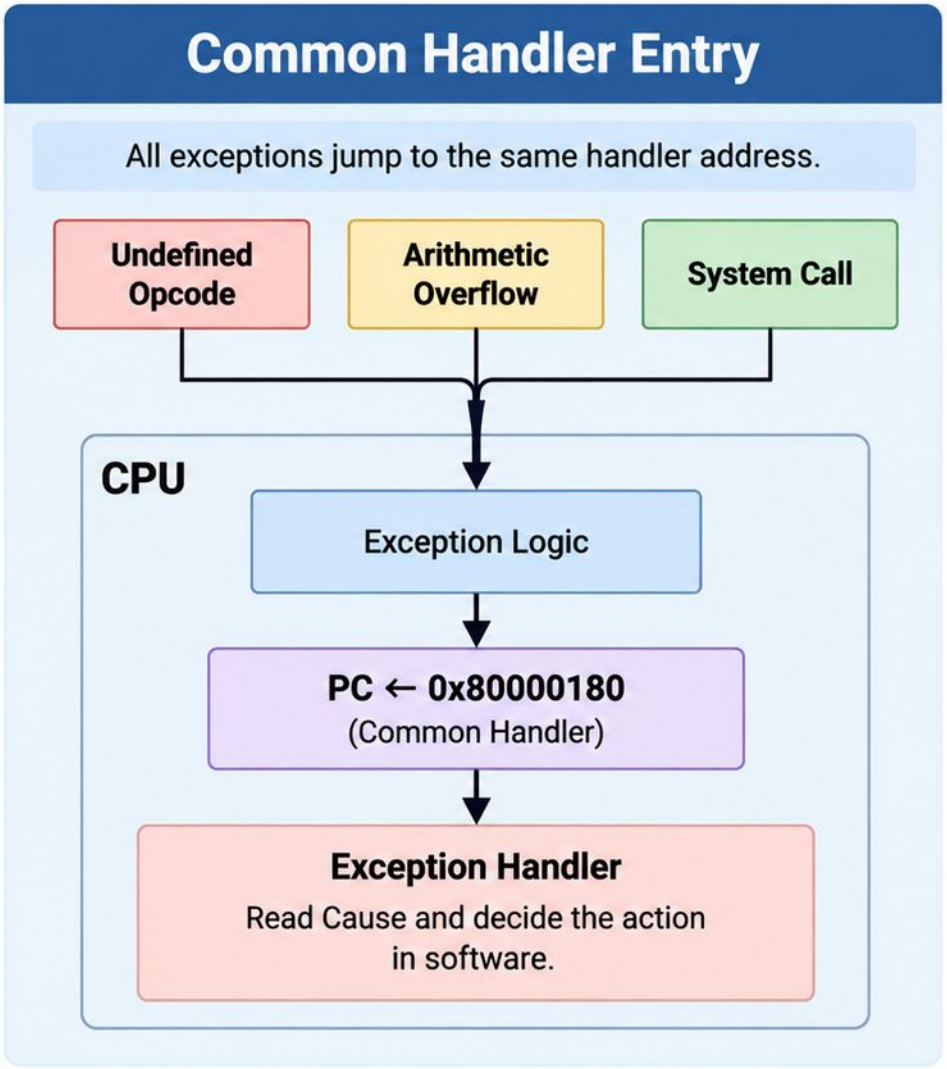
EPC = 0x00400028: Identifies the invalid instruction.

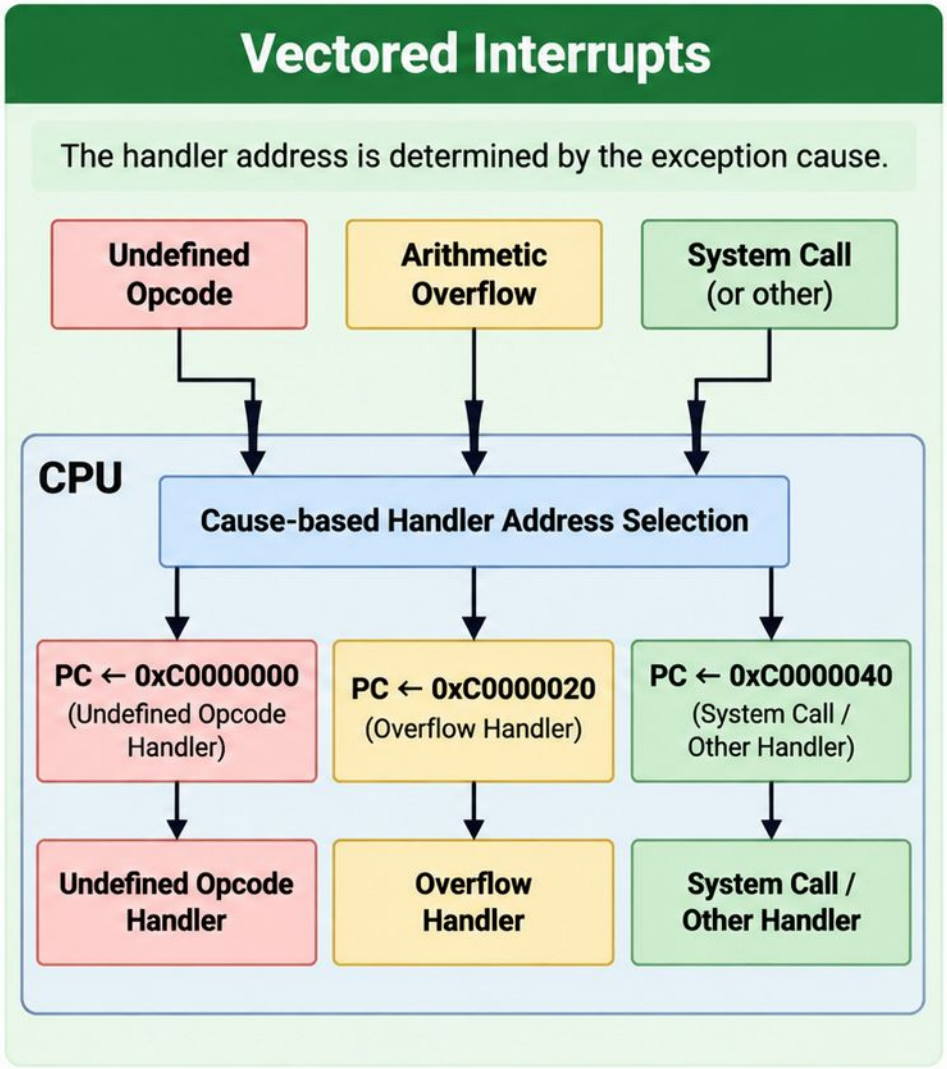
PC = 0x80000180: Execution transfers to the common exception handler.

An Alternate Mechanism

- Vectored Interrupts
  - Handler address determined by the cause
- Example:
  - Undefined opcode: C000 0000
  - Overflow: C000 0020
  - …: C000 0040
- Instructions either
  - Deal with the interrupt, or
  - Jump to real handler

Common vs. Vectored

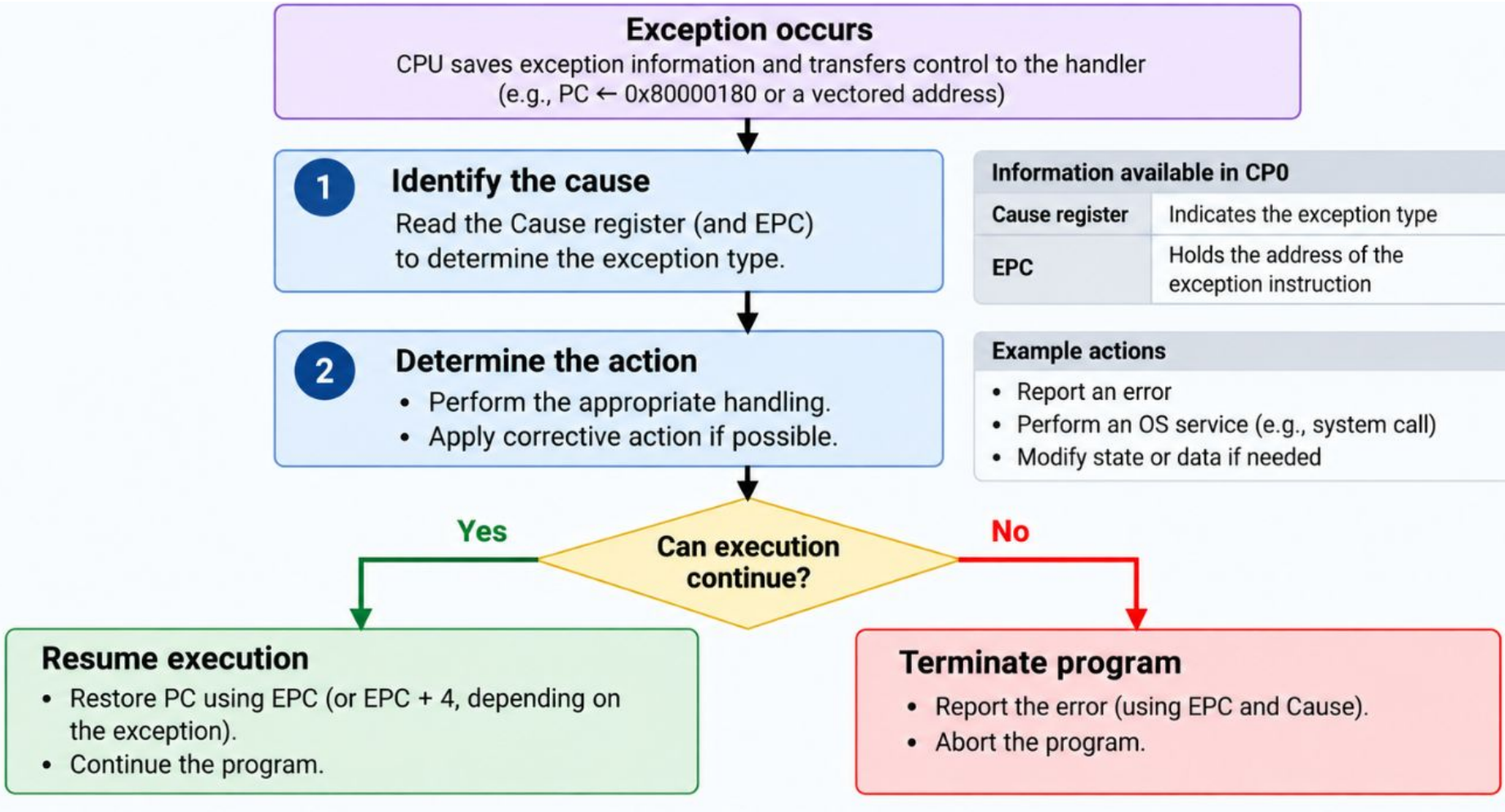




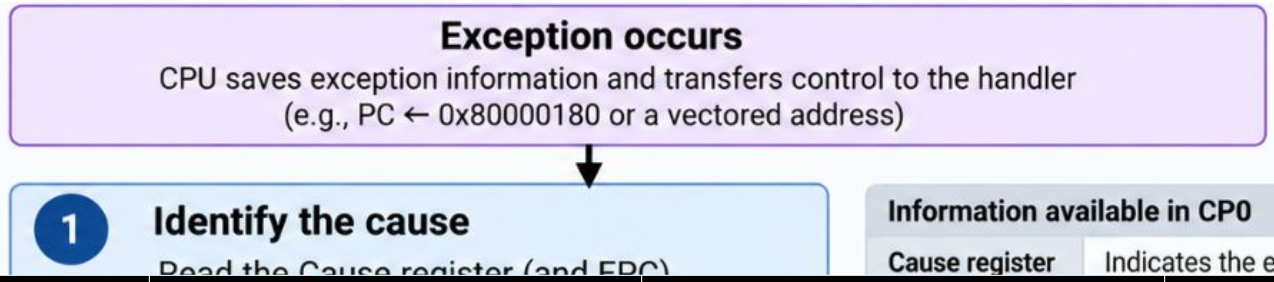
Handler Actions

- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
  - Take corrective action
  - use EPC to return to program
- Otherwise
  - Terminate program
  - Report error using EPC, cause, …

Handler Action Flow



Handler Action Flow



Handler Step	Undefined Opcode	Arithmetic Overflow	System Call
1. Identify Cause	Detect an invalid instruction.	Detect signed arithmetic overflow.	Detect a syscall instruction.
2. Determine Action	Report an invalid instruction error.	Report overflow or apply corrective action.	Perform the requested OS service.
3. Resume/Terminate	Usually terminate the program.	Terminate or resume if corrected.	Usually resume at the next instruction (EPC + 4).

Handler Action Flow

Q4. An exception handler encounters following situations:

Event	Exception Type	Handler Result
A	Undefined Opcode	The invalid instruction cannot be corrected.
B	Arithmetic Overflow	The handler successfully corrects the cause of overflow.
C	System Call	The requested OS service is completed.

Determine whether the handler should Restart, Continue, or Terminate the program for each event.

A4.

Event	Handler Action	Reason
A	Terminate	The invalid instruction cannot be corrected.
B	Restart	The corrected instruction can be re-executed.
C	Continue	Execution resumes after the syscall instruction.

Exception Properties

- Restartable exceptions allow execution to resum e after handling the exception.
- The EPC stores the address of the instruction as sociated with the exception.
- The exception handler determines whether exec ution can resume.
- If restartable, the CPU can return to the appropri ate instruction.
- Otherwise, the program may be terminated.

Exception Properties

1 EPC identifies the exception instruction

- When an exception occurs, the **EPC** register stores the address of the instruction that caused the exception.
- The handler uses **EPC** to locate the instruction.

EPC	0x00400020
Cause	Arithmetic Overflow
PC (in handler)	0x80000180

2 Not all exceptions are restartable

- Many exceptions are **restartable**: after corrective action, the same instruction can be re-executed.
- Some exceptions are **not restartable** and require terminating the program (e.g., undefined opcode).
- Restarting without fixing the cause may trigger the same exception again.

3 Resume PC depends on the handling result

- After the handler finishes, the processor resumes execution at an appropriate address.
- The new **PC** can be **EPC** (re-execute) or **EPC + 4** (continue to next instruction), depending on the exception type and the handler's action.
- The handler must select the correct return address.

Example Instruction Sequence

```
0x00400020  add  $t0, $t1, $t2   Overflow!
0x00400024  sub  $t3, $t4, $t5
0x00400028  lw   $t6, 0($t7)
```

Exception Handler
Read Cause and EPC
Perform appropriate action

Restart the instruction

PC ← EPC
0x00400020

Re-execute the instruction after fixing the cause (e.g., corrected operands).

Continue to the next instruction

PC ← EPC + 4
0x00400024

Skip the exception instruction and continue execution (e.g., system call).

- **EPC** records the address of the instruction associated with the exception.
- Only some exceptions are restartable.
- The handler chooses the appropriate return address (**EPC** or **EPC + 4**) based on the exception type and the handling result.

Exception Properties

Q5. A MIPS processor encounters an exception at address 0x00400048. The handler is located at 0x80000180. After handling the exception, determine the Resume PC for each situation:

Situation	Handler Result
A	The exception is corrected, original instruction must be re-executed.
B	A syscall is completed, the execution should continue
C	The exception cannot be corrected.

A5.

Situation	Resume PC	Explanation
A	0x00400048	Restart at EPC.
B	0x0040004C	Continue at EPC + 4.
C	No Resume PC	The program terminates.

Exception Example

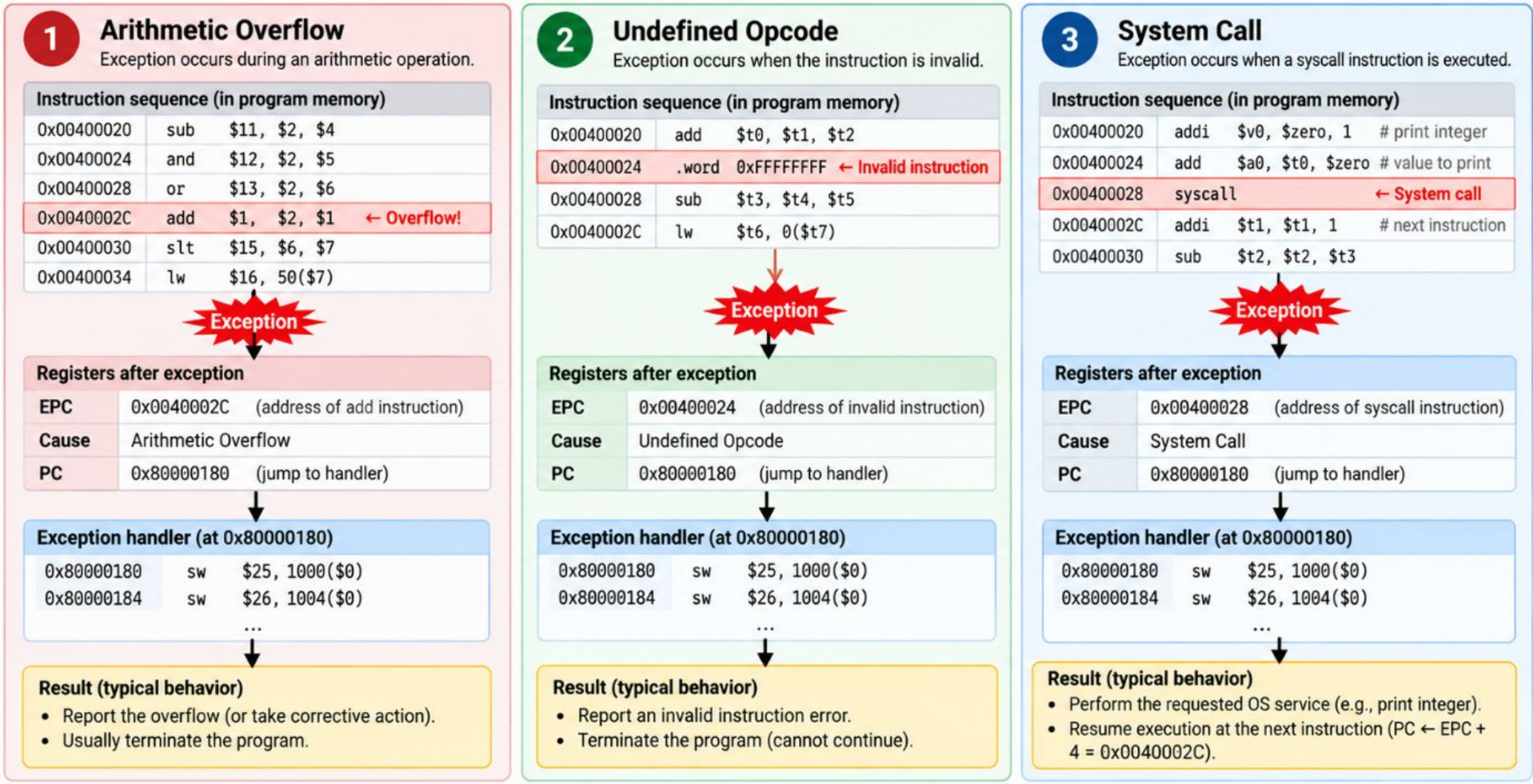
- Exception on add in (overflow)

```
40    sub   $11, $2, $4
44    and   $12, $2, $5
48    or    $13, $2, $6
4C    add   $1,  $2, $1
50    slt   $15, $6, $7
54    lw    $16, 50($7)
…
```

- Handler

```
80000180   sw    $25, 1000($0)
80000184   sw    $26, 1004($0)
…
```

Other Examples



Concluding Remarks

- The processor executes instructions through the fetch, decode, execute, and PC update steps.
- Combinational and sequential logic support data path operations and clocking.
- The datapath uses registers, the ALU, and mem ory to execute different instruction types.
- The control unit generates control signals to coor dinate datapath operations.
- The critical path determines clock performance, while exception handling manages unexpected e vents.

Section 3-6 Quick Questions

Q1. Given instruction delays, determine the minimum clock period and was ted time for an add instruction.

Q2. Classify arithmetic overflow, keyboard input, and syscall as Exceptions or Interrupts.

Q3. Given EPC, Cause, and PC values, identify the instruction that caused the exception.

Q4. Determine whether the handler should Restart, Continue, or Terminate for three exception scenarios.

Q5. Given EPC and handler results, determine the Resume PC for restarti ng, continuing, or terminating execution.

Exercise 1 — Single-Cycle Processor Performance

Two single-cycle MIPS processors, Processor A and Processor B, have the f ollowing instruction execution delays:

Instruction	Processor A	Processor B
add	400 ps	350 ps
beq	350 ps	300 ps
sw	650 ps	600 ps
lw	800 ps	700 ps

A program executes the following instructions:

Instruction	Count
add	4,000
beq	2,000
sw	1,000
lw	3,000

Assume both processors use a single-cycle datapath.

(a) Determine the minimum clock period of each processor.
(b) Calculate the total execution time of the program on each processor.
(c) Calculate the speedup of Processor B over Processor A.

Exercise 1 — Single-Cycle Processor Performance

Answer

(a) Clock Period

The clock period is determined by the slowest instruction (lw).

- Processor A: 800 ps
- Processor B: 700 ps

(b) Execution Time

Total Instruction Count = 4,000 + 2,000 + 1,000 + 3,000 = 10,000

Since CPI = 1:

- Processor A: 10,000 × 800 ps = 8 μs
- Processor B: 10,000 × 700 ps = 7 μs

(c) Speedup

Speedup = 8/7 = 1.143

Processor B is approximately 14.3% faster than Processor A.

Key Point: In a single-cycle processor, all instructions share the same clock period, which is determined by the slowest instruction.

Exercise 2 — Exception Handling Procedure

Consider the following MIPS instruction sequence:

Address	Instruction
0x00400020	add $t0, $t1, $t2
0x00400024	sub $t3, $t4, $t5
0x00400028	syscall
0x0040002C	lw $t6, 0($t7)
0x00400030	add $s0, $s1, $s2

Assume that an arithmetic overflow occurs while executing the s ub instruction. The common exception handler is located at 0x80 000180.

Describe the steps the processor takes to handle this exception, including EPC, Cause, PC, and the final program outcome.

Exercise 2 — Exception Handling Procedure

Answer

1. Detect Exception: Arithmetic overflow occurs at 0x00400024.
2. Save EPC: EPC ← 0x00400024.
3. Set Cause: Cause ← 1 (Arithmetic Overflow, using the simplified course encoding).
4. Redirect PC: PC ← 0x80000180.
5. Execute Handler: The handler reads Cause and determines the appropriate action.
6. Terminate Program: Assume the overflow cannot be corrected; execution does not resume.

Key Point: Exception handling saves the faulting instruction address in EPC, records the exception type in Cause, and redirects PC.

Exercise 3 — Complete Exception Execution Tracing

Consider the following MIPS instruction sequence:

Address	Instruction
0x00400020	addi $t0, $zero, 5
0x00400024	syscall
0x00400028	beq $t0, $zero, Exit
0x0040002C	addi $t0, $t0, -1
0x00400030	j Exit
0x00400034	Exit: add $s0, $t0, $zero

Assume the common exception handler is located at 0x80000180. The syscall service completes successfully, and the handler resumes the program. Assume no branch delay slots.

(a) Determine the values of EPC and PC when the syscall exception occurs.
(b) Determine the Resume PC after the handler completes.
(c) Trace the instruction addresses executed after returning from the handler until reaching Exit, and determine the final values of $t0 and $s0

Exercise 3 — Complete Exception Execution Tracing

Answer

(a) Exception Detection

- EPC = 0x00400024
- PC = 0x80000180

(b) Resume PC

Resume PC = EPC + 4 = 0x00400028

(c) Instruction Tracing

Address	Execution Result
0x00400028	beq not taken ($t0 = 5)
0x0040002C	$t0 = 5 - 1 = 4
0x00400030	Jump to Exit
0x00400034	$s0 = 4

Final values: $t0 = 4, $s0 = 4.

Key Point: After a completed syscall, execution resumes at EPC + 4. Subsequent branches and jumps determine the actual instruction execution sequence.